

对称密码算法的软件实现与优化

AES / SM4 / GIFT / TWINE 与 CTR / GCM / XTS

课程项目实施与实验报告

2026 年 7 月

摘要

本项目从统一的 C11 标量基线出发，实现 AES-128/192/256、SM4-128、GIFT-64 (128-bit key) 和 TWINE-80/128，并系统比较 T-table、shuffle/bitslice/fixslice 与新指令集。工作模式覆盖全部算法的 CTR，以及 AES/SM4 的 GCM、XTS；GCM 使用 ARM PMULL 加速 GHASH，XTS 同时支持 IEEE 1619 与 GB/T 17964-2021 的 tweak 乘法约定及 ciphertext stealing。Apple M2 Pro 上保存 3 次预热、15 次正式测量的全部样本；x86-64 路径只做完整交叉编译与反汇编确认，性能明确标记为待目标机实测。功能测试包括 1592 项 KAT/随机/边界检查和 3360 项 OpenSSL 3.6.2 差分检查。

关键词：对称密码；AES；SM4；GIFT；TWINE；T-table；NEON TBL；PMULL；GFNI；CTR；GCM；XTS

目录

1 材料、目标与交付边界	3
2 微架构基础与优化模型	3
2.1 吞吐率不是指令条数	3
2.2 端序与未定义行为	3
3 四种分组密码的基础实现	3
3.1 AES	3
3.2 SM4	4
3.3 GIFT-64 与 TWINE	4
4 统一接口与运行时分派	4
5 优化技术	4
5.1 T-table：空间换指令	4
5.2 Shuffle、bitslice 与 fixslice	4
5.3 新指令集	5

6	CTR、GCM 与 XTS	5
6.1	CTR	5
6.2	GCM	5
6.3	XTS	5
7	测试与工程验证	6
8	实验方法	6
9	结果与分析	6
10	安全、性能与平台权衡	9
11	复现方式	9
12	结论	10

1 材料、目标与交付边界

课程材料为 86 页《对称密码的软件实现与优化》。原始 PPT 的项目副本 SHA-256 为

```
69b70bead7ddf5feb3f12d582c5770c43
241110eda6f45f9670cd4f18fc903cb
```

该值与用户提供文件逐字节一致；逐页主题与代码/实验落点见 `materials/SLIDE_MAP.md`。材料从微架构、寄存器、T-table 讲到 `shuffle`、复合域、AES/SM4 指令、GIFT/TWINE `fixslicing` 与 CTR/GCM/XTS，因此项目也按“基础实现 → 表优化 → SIMD/新指令 → 模式优化”组织。

安全边界：本项目用于教学和可复现实验，不是认证密码库。尤其 T-table 会以秘密值索引缓存，不应成为真实系统的默认后端。

2 微架构基础与优化模型

2.1 吞吐率不是指令条数

现代处理器可多发射并行执行，实际耗时由执行端口、依赖链、指令时延和吞吐共同决定。因而同样的轮函数可以有三类改进：

1. 缩短关键依赖链，例如把互不相关的分组交错执行；
2. 减少内存层次开销，例如把状态保持在寄存器，或缩小表；
3. 用一条 SIMD/密码指令替代多条标量指令。

全展开可以删除循环控制，却会扩大指令缓存占用。本项目保留清晰的循环基线，只在数据表示和后端处做可测量的变换，同时以对象尺寸 CSV 记录代码代价。

2.2 端序与未定义行为

所有外部字节串都通过 `sc_load_be32/64` 与 `sc_store_be32/64` 显式转换。2 KiB SM4 重叠表虽然利用偏移读取旋转后的 32-bit 字，却不把非对齐地址强制转换成 `uint32_t*`，从而避免别名和对齐未定义行为。这一点由 ASan/UBSan 与 `-Werror` 共同约束。

3 四种分组密码的基础实现

3.1 AES

AES 按 FIPS 197 实现 128/192/256-bit 密钥扩展、SubBytes、ShiftRows、MixColumns 及逆变换。参考路径以 16 字节状态表示；T-table 同时实现正向与逆向四表。ARM 路径用 AESE/AESMC 与 AESD/AESIMC，x86 路径用 AES-NI，多块时可进入 VAES。所有路径使用同一轮密钥并与 FIPS KAT 交叉比较。

3.2 SM4

SM4 使用 32 轮广义 Feistel 更新：

$$X_{i+4} = X_i \oplus L(\tau(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)),$$

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24).$$

密钥扩展和数据轮均依据 GB/T 32907-2016。实现把输入、输出与轮密钥端序明确分开，并用官方向量 012345...3210 $\rightarrow$ 681edf...4246 验证。

3.3 GIFT-64 与 TWINE

GIFT-64 采用四路 bitslice 状态，S 盒直接变为四个位平面上的布尔网络，位置换由固定掩码网络完成；批量 API 每次处理四个独立分组。TWINE 以 16 个 nibble 表示状态，分别实现 80-bit 与 128-bit 密钥调度；KAT 密文为 7c1f0f80b1df9c28 与 979ff9b379b5a9b8。

4 统一接口与运行时分派

公共上下文 `sc_ctx` 对外保持不透明，提供密钥初始化、单块/多块加解密、块长/算法/后端查询和明确错误码。单块 API 支持原地操作。后端可选 `ref`、三种 `ttable`、`shuffle`、`aes-hw`、`gfni`、`sm4-hw` 与 `auto`。

Listing 1: 统一接口的核心调用

```
sc_ctx ctx;
sc_status rc = sc_init(&ctx, SC_AES_128, SC_BACKEND_AUTO,
                      key, sizeof(key));
rc = sc_encrypt_blocks(&ctx, input, output, blocks);
```

ARM64 检测 AES、PMULL 与 SM4 能力；x86 检测 AES-NI、SSSE3、AVX2、VAES、PCLMUL、VPCLMUL 和 GFNI。显式请求不可用 ISA 时返回 `SC_ERR_UNSUPPORTED`，不做静默冒充。

5 优化技术

5.1 T-table：空间换指令

表越小不保证越快：1 KiB 路径增加旋转，2 KiB 路径增加非对齐字节组装；4 KiB 路径则给 L1 数据缓存更大压力。四条路径同时存在，性能结论交给同一基准而不是先验判断。

5.2 Shuffle、bitslice 与 fixslice

TWINE 的 4-bit S 盒天然适合 16-byte TBL：一条 TBL 并行完成 16 个 nibble 查找，第二次 TBL 完成轮置换。SM4 的 8-bit S 盒不能直接由单个 16 项表表示，本项目使用 16 行固定表和掩码选择，访问轨迹与秘密无关；这是常量时间教学路径，但单块时指令数高。GIFT 的布尔 S 盒和固定 bit permutation 完全不查秘密索引表，更接近 bitslice/fixslice 思路。

表 1: 表布局与数据路径

算法/后端	表大小	每轮方法
AES 四表	4 KiB	SubBytes+ShiftRows+MixColumns 合并
SM4 四表	4 KiB	四个字节分别查表后异或
SM4 单表	1 KiB	查一个表, 再旋转 8/16/24 bit
SM4 重叠	2 KiB	8 字节重复项, 偏移 0/3/2/1 读取

5.3 新指令集

- **AES:** ARM AESE/AESD 为本机四路运行路径; x86 AES-NI 与 VAES 已进入四路/八路实际后端并通过反汇编确认。
- **GHASH:** PMULL 做两个 64-bit 多项式的 Karatsuba 乘法, 再以 $x^{128} = x^7 + x^2 + x + 1$ 两次折叠约减; x86 PCLMUL/VPCLMUL 已进入实际 GHASH 后端。
- **GFNI:** SM4 S 盒按 $Y = A_1X + C_1$, $S(X) = A_2Y^{-1} + C$ 分解。标量 GFNI 模型穷举 256 个输入并逐项等于标准 S 盒, x86 源码用两条 affine/affine-inverse 指令。
- **AES 辅助 SM4:** 先用固定扫描预映射到 AES 逆 S 盒值, 再以 AESE/AESENC 完成四块 SM4 S 盒并逆转 ShiftRows; 全程不以秘密值直接索引缓存。
- **专用 SM4:** ARM SM4E/SM4EKEY 与 Intel VSM4RND4/VSM4KEY4 已接入运行后端并检查实际函数; M2 Pro 不支持 SM4 指令, 运行时跳过。

6 CTR、GCM 与 XTS

6.1 CTR

CTR 对四类密码全部开放。计数器按分组宽度大端递增, 每批生成八个计数器并调用多块 API; 末尾不足一块时只异或所需字节。调用前先验证全部计数器空间, 回绕时不修改输出, 并返回错误码 SC_ERR_COUNTER_WRAP。

6.2 GCM

GCM 只接受 128-bit 分组的 AES/SM4。96-bit IV 直接形成 $J_0 = IV \parallel 0^{31} \parallel 1$, 其他 IV 先经 GHASH。AAD、密文和长度字段均按 SP 800-38D 处理。解密先以常量时间比较标签; 失败返回 SC_ERR_AUTH 并清零输出。NIST AES-GCM 与 RFC 8998 SM4-GCM 向量均纳入回归。GHASH 预计算 H^2, H^3, H^4 , 完整的四块组用四个无数据依赖的乘法聚合。

6.3 XTS

XTS 使用两个独立密钥上下文, 拒绝短于 16 字节的数据单元, 并对非整块尾部执行 ciphertext stealing。IEEE 1619 约定在每个字节内左移并以 0x87 约减; GB/T 17964-2021 约定在每个字节内右移并以 0xe1 约减。OpenSSL 3.6 的 SM4-XTS 默认值为 GB, 本项目同时保留 IEEE 选项。普

通分组按八路预生成 `tweak` 并批量执行 XEX；CTS 在写回前保存尾部，因而支持原地加解密。

7 测试与工程验证

表 2: 验证层次

层次	内容
KAT	AES 三种密钥、SM4、GIFT-64、TWINE-80/128、NIST GCM、RFC 8998
内部交叉	参考/表/shuffle/硬件后端，固定种子随机输入，原地与非对齐语义
模式边界	CTR 尾部/回绕、GCM AAD/错误标签清零、XTS CTS/最小数据单元
OpenSSL	3360 项 AES/SM4 ECB、CTR、GCM、XTS 随机差分；AES/SM4 CTS
工具链	-Wall -Wextra -Wpedantic -Werror、ASan、UBSan
ISA	ARM AESE/TBL/PMULL/SM4E； x86 AES/VAES/PSHUFB/GFNI/CLMUL/VSM4

8 实验方法

实验平台为 Apple M2 Pro (Darwin arm64)，编译器为 clang-17.0.0 (clang-1700.0.13.5)。消息长度取 64 B、512 B、4 KiB、8 KiB、64 KiB、1 MiB。每组预热 3 次、正式运行 15 次；每个样本至少处理 256 KiB，记录完整纳秒值，汇总 median、Q1、Q3、IQR、ns/byte 和十进制 GB/s。密钥扩展在计时区间外，模式的计数器、tweak、GHASH 和标签均在计时区间内。volatile checksum 防止编译器删除计算。

x86-64 结果文件中的 cycles/byte 为 null；当前仅完成源码交叉编译、静态库生成和目标指令确认，不把 ARM 主机数据改名为 x86 数据。

9 结果与分析

在 1 MiB 数据上，AES 参考、四表和硬件后端分别为 0.134、0.199、8.812 GB/s。T-table 相对参考为 1.48×，AESE/AESMC 为 65.72×。SM4 四表由 0.066 提升至 0.097 GB/s，即 1.46×。1 KiB 单表非常接近四表，2 KiB 重叠读取在本机反而较慢，说明“更小表”必须与额外旋转/装配成本一起评价。

硬件 AES 在 64 B 时已经领先，随后趋于稳定；本实验把函数调用和循环调度算入短消息成本。SM4 shuffle 每轮固定扫描 16 行表，并以四块并行摊薄装配开销；其目标是展示无秘密相关访存，而不是声称单块最高性能。TWINE TBL 与参考接近，说明仅替换 S 盒还不足以自动获得多块并行收益。

AES-CTR 能直接发挥硬件块加密能力；XTS 还要执行第二个密钥的 tweak 加密与每块乘 α ，因此低于 CTR。GCM 已用 H^2-H^4 消除四块组内的串行依赖，跨组仍保留必要的 GHASH 依赖。

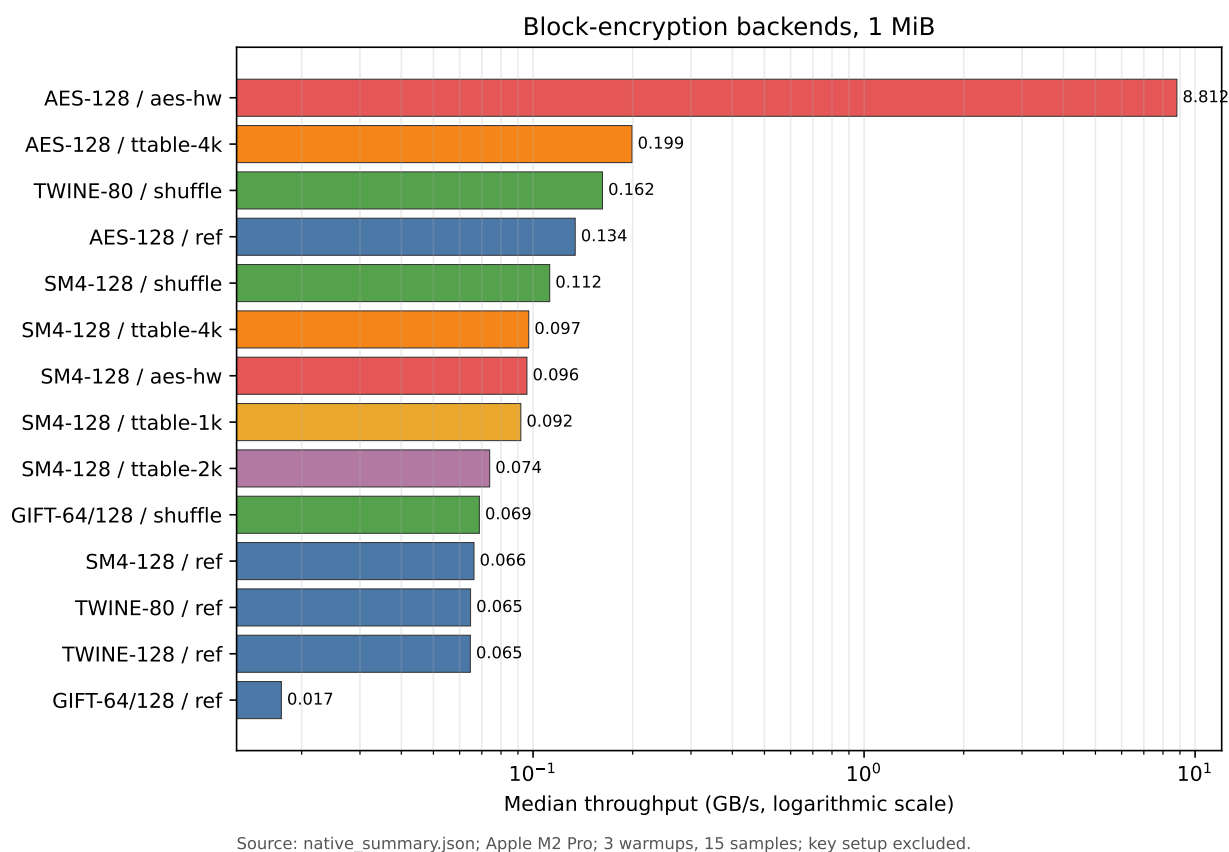


图 1: 1 MiB 分组加密吞吐率。横轴为对数尺度，数据由汇总 JSON 生成。

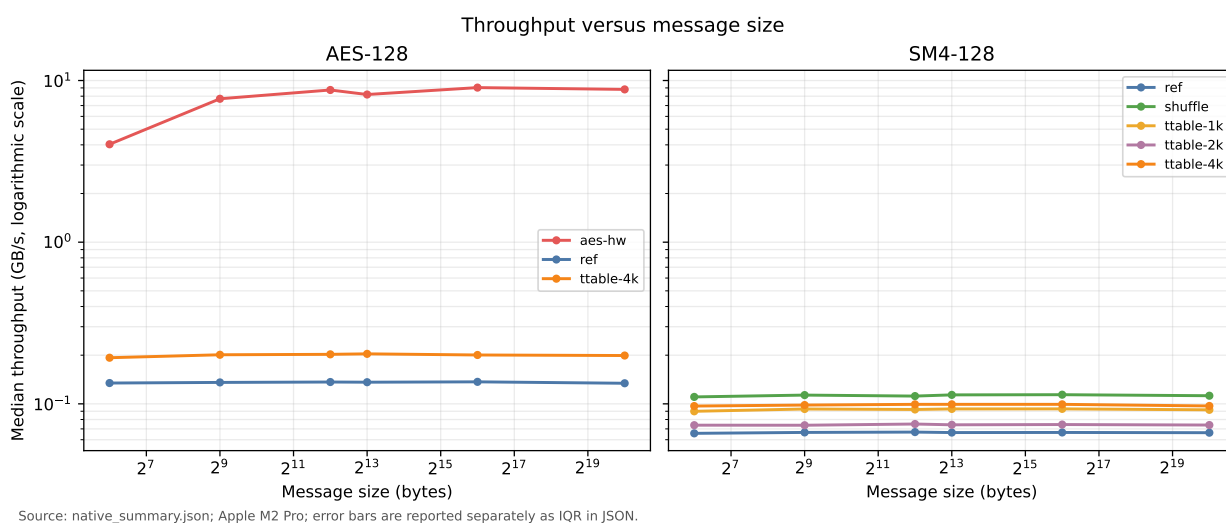


图 2: 不同消息长度下 AES/SM4 后端的稳态吞吐。

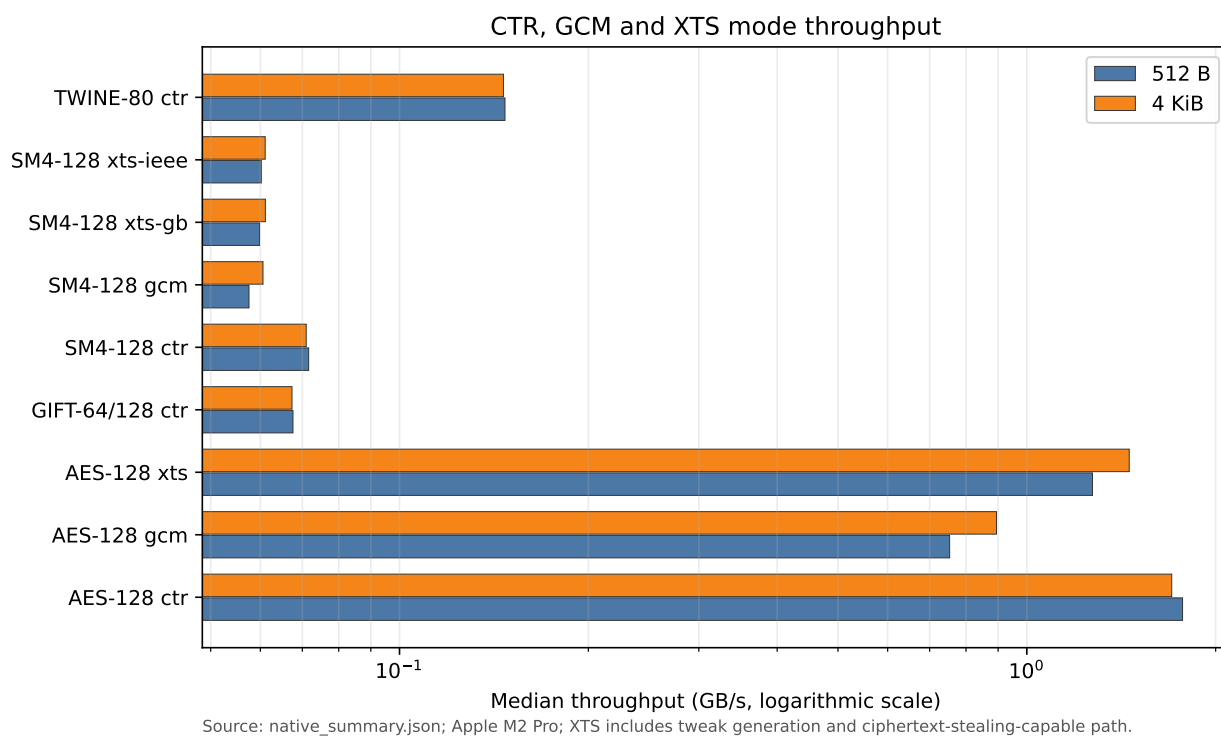


图 3: 512 B 与 4 KiB 的 CTR/GCM/XTS 吞吐量。

IEEE 与 GB 两种 SM4-XTS 吞吐相近，符合两者只改变轻量 tweak 乘法的预期。

表 3: 1 MiB 全部实测结果（直接由 JSON 生成）

算法	后端	操作	GB/s	ns/byte	IQR
AES-128	ref	block	0.134	7.459	0.098
AES-128	ttable-4k	block	0.199	5.025	0.113
AES-128	aes-hw	block	8.812	0.113	0.009
AES-128	aes-hw	block-dec	8.128	0.123	0.004
SM4-128	ref	block	0.066	15.082	0.092
SM4-128	ttable-4k	block	0.097	10.305	0.174
SM4-128	ttable-1k	block	0.092	10.888	0.085
SM4-128	ttable-2k	block	0.074	13.522	0.156
SM4-128	shuffle	block	0.112	8.907	0.085
SM4-128	aes-hw	block	0.096	10.435	0.121
SM4-128	ttable-2k	block-dec	0.073	13.618	0.071
GIFT-64/128	ref	block	0.017	57.591	0.320
GIFT-64/128	shuffle	block	0.069	14.528	0.121
TWINE-80	ref	block	0.065	15.445	0.108
TWINE-80	shuffle	block	0.162	6.169	0.073
TWINE-128	ref	block	0.065	15.469	0.153
AES-128	aes-hw	ctr	1.727	0.579	0.016

算法	后端	操作	GB/s	ns/byte	IQR
SM4-128	ttable-2k	ctr	0.071	14.142	0.156
GIFT-64/128	ref	ctr	0.017	58.270	0.222
GIFT-64/128	shuffle	ctr	0.067	14.958	0.100
TWINE-80	shuffle	ctr	0.146	6.842	0.144
AES-128	aes-hw	gcm	0.913	1.095	0.029
AES-128	aes-hw	gcm-dec	0.880	1.136	0.038
SM4-128	ref	gcm	0.058	17.268	0.959
AES-128	aes-hw	xts	1.429	0.700	0.034
AES-128	aes-hw	xts-dec	1.387	0.721	0.026
SM4-128	ref	xts-ieee	0.061	16.310	0.204
SM4-128	ref	xts-gb	0.061	16.361	0.108
SM4-128	ref	xts-gb-dec	0.061	16.348	0.076

10 安全、性能与平台权衡

1. **T-table**: 通常减少指令，却泄露秘密相关缓存地址；只用于教学。
2. **Shuffle/bitslice**: 访问模式固定，更适合常量时间；需要足够多并行块才能摊薄转置和数据装配。
3. **硬件密码指令**: AES/PMULL 兼顾性能和固定执行路径，但仍不能替代协议级 nonce 管理、密钥生命周期和故障防护。
4. **运行时分派**: 可移植二进制必须检查 CUID/XGETBV 或 ARM feature; “编译器接受 intrinsic” 不等于当前 CPU 可以执行。
5. **x86 平台**: VSM4 等 2026 扩展已验证编码，但没有目标硬件，所以报告不提供伪造吞吐和 cycles/byte。

11 复现方式

```
make test
make test-sanitize
make check-x86
make bench
make report
```

数据链完全由文件驱动：

- 原始样本：results/raw/native_samples.csv;
- 汇总统计：results/summary/native_summary.json;
- 图表脚本：scripts/plot_results.py;
- 数值宏脚本：scripts/generate_report_data.py。

因此每个数字都能回溯到正式样本。

12 结论

本项目建立了从正确性基线到多类优化和三种工作模式的完整链条。实测说明：表优化的收益受缓存与额外指令共同约束；硬件 AES 的数量级优势最稳定；shuffle/fixslice 必须和多块数据布局一起设计；GCM/XTS 不能只优化分组密码而忽略 GHASH、计数器和 tweak。ARM64 已完成原生验证，x86 已完成可审计的编译/指令证据，下一步只需在真实 x86 主机复用同一基准生成 cycles/byte，无需修改报告的数据来源原则。

参考文献

- [1] NIST, *FIPS 197: Advanced Encryption Standard*, 2023. <https://csrc.nist.gov/pubs/fips/197/final>
- [2] 国家标准化管理委员会, *GB/T 32907-2016 SM4 分组密码算法*. <https://openstd.samr.gov.cn/bzgk/std/newGbInfo?hcno=7803DE42D3BC5E80B0C3E5D8E873D56A>
- [3] Banik et al., *GIFT: A Small Present*, CHES 2017. <https://www.iacr.org/archive/ches2017/10529227/10529227.pdf>
- [4] Suzaki et al., *TWINE: A Lightweight Block Cipher for Multiple Platforms*, SAC 2012. https://jpn.nec.com/rd/tg/dss/pdf/twine_SAC_full_v5.pdf
- [5] NIST, *Block Cipher Modes*. <https://csrc.nist.gov/projects/block-cipher-techniques/bcm>
- [6] P. Yang, *RFC 8998: ShangMi Cipher Suites for TLS 1.3*, 2021. <https://www.rfc-editor.org/rfc/rfc8998.html>
- [7] W. Guo, *Efficient Constant-Time Implementation of SM4*, 2022/1154. <https://eprint.iacr.org/2022/1154.pdf>
- [8] OpenSSL, *EVP_EncryptInit: xts_standard*. https://docs.openssl.org/3.4/man3/EVP_EncryptInit/